Expert Systems with Applications 37 (2010) 4850–4859

Contents lists available at ScienceDirect

Expert Systems with Applications

journal homepage: www.elsevier.com/locate/eswa

Computationally efficient behaviour based controller for real time car racing simulation

C.H. Tan, K.C. Tan *, A. Tay

Department of Electrical and Computer Engineering, National University of Singapore, 4 Engineering Drive 3, Singapore 117576, Singapore

ARTICLE INFO

Keywords:
Behaviour based controller
Genetic algorithms
Simulated car racing

ABSTRACT

This paper examines the design of a controller that is computationally efficient yet demonstrates highly competitive performance for a real time simulated car racing game. In turn based games, the game artificial intelligence (AI) is able to compensate for its lack of game reasoning by evaluating board positions millions of times faster than the human player. However, such extreme resource requirements are impractical for fast paced and real time games, i.e. racing games, sports simulators, first person shooters and real time strategy games. This paper proposes and describes in detail an evolved behaviour based controller that combines the good response time of behaviour based systems and search capability of evolutionary algorithms to evolve competitive driving skills for a real time car racing game. The proposed controller is tested against the top five participants of the Simulated Car Racing Competition held during the 2007 IEEE Congress on Evolutionary Computation (CEC) to evaluate its generalization performance against previously unseen controllers. The proposed behaviour based controller is able to outperform all its opponents in direct competition, and is also the most computationally efficient.

© 2009 Elsevier Ltd. All rights reserved.

1. Introduction

The quality of commercial computer games is directly related to their entertainment value (Tozour, 2002). Game artificial intelligence (AI), being an essential part of a game, has become an important selling point of games (Forbus & Laird, 2002). In this aspect, games developers compete with one another by creating more sophisticated and intelligent game AI to offer better game play experiences. However, the current state of game AI is still, in general, of low quality (Schaeffer, 2001). There is a general dissatisfaction among game players with the level of the artificial intelligence of computer controller opponents. This leads to players preferring to play against human controlled opponents (Schaeffer, 2001), via hot seat, split screen, local network, Bluetooth, infrared and most prominently the Internet. Such player preferences have also partly contributed to the boom in the development of massively multiplayer online games in recent years. However, in situations where human game partners are unavailable, a competent game AI is still desirable.

World class game AI does exist and many have been developed that are able to beat good human players (Lucas & Kendall, 2006). But these are generally restricted to slower paced, turn based perfect information games (Buro & Furtak, 2003) such as Deep Blue for International Chess (IBM Research, 2007) and Chinook for Checkers (Schaeffer, 1997). Deep Blue compensates for its lack of game reasoning by evaluating board positions millions of times faster than the human player. However, such extreme resource requirements are impractical for commercial games where majority of the CPU time and memory is allocated to graphics instead of AI. For faster paced, real time type of games, i.e. racing games, sports games, flight simulators, first person shooters (FPS) and real time strategy (RTS), such brute force evaluation methods are not feasible. Computationally efficient methods that do not compromise in performance are necessary requirements for the implementation of game AI in such real time games (Spronck, Ponsen, Sprinkhuizen-Kuyper, & Postma, 2006).

Behaviour based artificial intelligence (BBAI), popular in the field of robotics, provides some inspiration to address these real time computational requirements. In this methodology (Brooks, 1986), intelligence is perceived as a large number of relatively simple and robust modular components. Each of these components work only within a specific set of conditions which it can identify from the environment. BBAI is reactive in nature and operates without search or deliberation, and is therefore very successful in time critical applications such robotics and interactive virtual reality (Sengers, 1998). However, the disadvantage of reactive intelligence is in its design process since individual components need to be designed by hand.

Fortunately, computational intelligence (CI) techniques such as neural network, fuzzy logic and evolutionary computation have

* Corresponding author.
E-mail addresses: g0600100@nus.edu.sg (C.H. Tan), eletankc@nus.edu.sg (K.C. Tan), eletaya@nus.edu.sg (A. Tay).

0957-4174/$ - see front matter © 2009 Elsevier Ltd. All rights reserved.
doi:10.1016/j.eswa.2009.12.030

been demonstrated to be a valuable tool that can be employed to simplify the process of designing controller behaviours and optimizing its related parameters. Neural networks were trained as evaluation functions for checkers (Chellapilla & Fogel, 2001). Evolutionary algorithms had been applied to evolve competent players and to analyze results in "Prisoner's Dilemma" games (Quek & Goh, 2007; Ghoneim, Abbass, & Barlow, 2008). Hybrid fuzzy logic methodologies had been applied to incomplete information resource allocation games and network flow board games (Braathen & Sendstad, 2004). Genetic programming was applied to learn tactical behaviours by observing how human players perform in a driving simulator (Fernlund, Gonzalez, Georgiopoulos, & DeMara, 2006). Game agent controllers employing CI techniques had been successfully applied to games such as racing games (Chaperot & Fyfe, 2006; Togelius & and Lucas, 2006a, 2006b), soccer simulation (Nakashima, Udo, & Ishibuchi, 2004; Nakashima, Takatani, Udo, & Ishibuchi, 2004; Stone, Sutton, & Kuhlmann, 2005), role playing games (Spronck, 2005), maze chase games (Yannakakis & Hallam, 2004; Yannakakis & Hallam, 2005) and real time strategy games (Bryant & Miikkulainen, 2003; Miikkulainen et al., 2006; Stanley, Bryant, & Miikkulainen, 2006) with reasonable performance against an average human player. While complex behaviours cannot be reliably and predictably evolved, simpler behaviours can be quickly found and thoroughly exploited. This characteristic suitably complements the process of designing the simple modules used in behaviour based controllers.

This paper examines the design of a computationally efficient controller for controlling a car in a real time simulated car racing game by using a hybridization of behaviour based design and evolutionary computation search. The proposed controller will be referred to as *behaviour based controller* for the remaining of this paper. The behaviour based controller will evaluated and compared based on two metrics, computational efficiency and competitive performance. The best evolved controller will then be tested against the top five participants from the Simulated Car Racing Competition held during 2007 IEEE Congress on Evolutionary Computation (CEC) in order to benchmark its generalization performance against previously unseen controllers.

This paper is organized into the following sections: Section 2 provides a brief description of the simulated car racing game. Section 3 presents the design and components of the behaviour based controller. Section 4 details the experiments conducted on the behaviour based controller and the analysis of its results. Section 5 presents the results and discussion from testing the behaviour based controller against previously unseen controllers. Finally, conclusions are drawn in Section 6.

2. Simulated car racing

This section describes the car racing simulator (Togelius & Lucas, 2005) and racing game used as the problem domain in this paper. In this game, two players drive their cars in an open arena and earn points by driving through an ordered sequence of waypoints. The player with more points at the end of the stipulated game time wins the game. An illustration of the game arena is shown in Fig. 1.

The dimension of the competition field is 400 pixels by 400 pixels and is not occupied by any walls or obstacles. As such, vehicles are free to drive outside of the competition field. However, only the competition field is visible to the human controlled player. Hence, this setup is advantageous to computer controlled cars as their sensors will continue to function even if their current position is outside the game field.

The objective of each race is to pass as many waypoints as possible within an allotted time. Two waypoints are visible on the competition field, the current and the next. However, the next

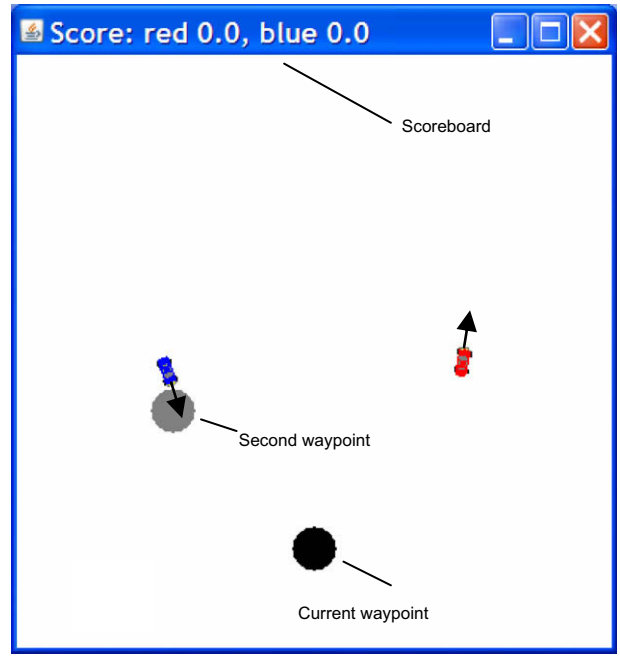

Fig. 1. The simulated car racing game arena.

waypoint cannot be collected unless the current is passed. That is, the waypoints must be collected in an ordered sequence. Nothing will happen when a car drives through the next waypoint. Whenever the current waypoint is passed, the car that passes it gain 1 point, the next waypoint will become the current and a new waypoint will be generated to replace the next. The method of generating waypoints in this model differs from that of the original which was generated randomly around the field. The main disadvantage of randomly generated waypoints is that two or more sequential waypoints may be generated in close proximity of one another and in severe cases, even overlapping. This results in a particular vehicle gaining two or more points in a single approach. This can be viewed as a biased allocation of points especially in a race where collecting waypoints is its main objective. As such, the following method of generating waypoints is proposed. The locus of the $(k + 1)$th waypoint always lies on the circumference of a circle of radius (400/3) pixels centered on the kth waypoint within the visible competition area. The arbitrarily chosen radius is to ensure waypoints do not overlap while maintaining some distance between waypoints to allow opposing vehicles an opportunity to mount a viable counter strategy. In addition, the very first waypoint is always initialized on the locus of a vertical straight line running through the center of the field. This is to ensure that the initial condition is not in favor of any vehicle in particular.

The vehicles themselves are controlled using digital trigger type controllers like the directional controls found on console game control pads. The four distinct on-off control signals, accelerate, decelerate, left and right turn combine to form a total of nine possible controller states, inclusive of a neutral state where no key is asserted. The controller can take inputs either from the keyboard or an AI algorithm. The included physics model is reasonably detailed, allowing for collisions between vehicles as well as side skidding. A technically skilled controller, whether human or not, will be able to execute such maneuvers when cornering to their advantage.

When driving towards the current waypoint, care must be taken not to approach it at too high a speed. A driver that is accelerating and steering towards a waypoint may overshoot the waypoint and end up orbiting around it. Additionally, if a driver overshoots the current waypoint, it may be put at a disadvantage

if the next waypoint is positioned behind the car. Driving at a slower speed may help in most situations but runs the risk of losing the current waypoint to the opposing driver and therefore wasting valuable time driving towards a waypoint that it is unable to win.

Using a more aggressive approach, the driver may choose to intentionally collide with the opposing driver in the hope of throwing its opponent off course and thereby increasing its own chances of arriving at the current waypoint first. Alternatively, the driver may choose to throw itself in the path of its opponent to block its path and perhaps get a helpful bump towards the current waypoint. However, this requires the driver himself to be competent in recovering from collisions and also to predict the most likely outcome of the collision to determine whether or not it is advantage to do so.

From a more tactical point of view, the driver can try to predict which driver will reach the current waypoint first. Given the situation, the driver can choose to drive faster towards the current waypoint or drive towards the next waypoint and wait there instead. This way, the driver loses a point for the current waypoint but wins the next point once it becomes activated and all is square for the race to the next waypoint after that. However, predicting which driver will reach the current waypoint first requires a good understanding of the game dynamics as well as the driving behaviour of the opposing driver. Conversely, the opposing driver may also decide to forsake the current waypoint, in which case it becomes more logical to drive towards the current waypoint now. In the example shown in Fig. 1, both cars are roughly equidistance from the current waypoint. The red car is on the right and the blue car is on the left. However, the red car is facing away from the waypoint while the blue car is facing the waypoint directly. The red car can choose to reverse towards the waypoint to avoid wasting time to make a U-turn. But if the red car knows that the reversing acceleration is slower than the forward acceleration, it will know the race to this waypoint is lost. In such a situation, the red car should forgo this waypoint and drive forward towards the next waypoint instead. Additionally, the blue car, after driving through the current waypoint, will likely be back facing the next waypoint and hence be in a poor position is win the next point.

Two heuristic controllers with basic driving behaviours packaged in the car racing simulator (Togelius & Lucas, 2005) are used in this paper and will be described in the following subsections. The HeuristicSensibleController is also used as the main training opponent in this paper as it represents a well tuned naïve driver that places sufficient selection pressure on the population during training.

2.1. HeuristicSensibleController

The HeuristicSensibleController (HSC) only drives forwards and will steer itself towards the current waypoint depending on whether it is on the left or the right. It drives with an upper speed limit of 7 pixels per time step which is considered a moderate speed. If the instantaneous speed falls below the speed limit, it exerts the accelerate command but if it is above the limit it simply issues the neutral driving command. One of the disadvantages of this controller is its disregard for the minimum turning radius. Because of this, it sometimes ends up orbiting around a waypoint that is too near to its side. Another drawback of this controller is that it does not have any waypoint prediction mechanisms. That is, it simply drives towards the current waypoint, disregarding whether or not it will reach the current waypoint before its opponent.

2.2. HeuristicCombinedController

The HeuristicCombinedController (HCC) is a more complex controller when compared to the HSC. Its behaviour will change depending on whether or not its present position is nearer to the current waypoint than the other car. If it is nearer, it behaves identically to the HSC. However, if it is further away, the controller activates an "underdog" mode and drives towards the next waypoint instead, stopping in the vicinity of the next waypoint. In underdog mode, its speed limit is proportionate to the distance towards the next waypoint. This controller reduces to the HSC in solo races.

3. Behaviour based controller

The proposed controller design is inspired by the behaviour based design methodology to take advantage of its computation efficiency. In this section, the behaviour based AI (BBAI) methodology will be briefly described followed by detailed discussion on the various components and performance of the behaviour based controller.

In BBAI, intelligence is made up of a large number of modular components which are relative simple and robust. Each of these components work within a specific set of conditions which it is able to observe from the environment. These components are organized into layers in a hierarchy which are able to interact with one another. The constraints here are that no components will have access to another's internal states but it is possible to observe their inputs and outputs. Additionally, a higher layer may subsume a lower layer by affecting its inputs and outputs. This is known as the subsumption architecture (Brooks, 1986).

This methodology was powerful because of its simplicity and robustness. Each individual component encodes only a simple behaviour such as moving forward, turning or avoiding objects, thus can be programmed reliably. In the original implementation, there was no memory or learning in its architecture, hence the resultant was that of a reactive behaviour. This turns out to be an advantage for BBAI because it is computationally efficient and hence suitable for systems that require good response time such as in the design of smart robots (DeSouza & Kak, 2004) and also in the real time racing game in this paper.

In the behaviour based controller, the basic driving behaviours of the car racing controller such as accelerating, braking and steering are organized at the lowest level. A separate component for the prediction of waypoints is placed at a higher level so that it is able to augment the input of the driving layer in order to dictate which waypoint is more advantageous for the car to drive towards.

The disadvantage of reactive intelligence is its design process because it performs no search or learning by itself (Bryson, 2002) and all behaviours must be designed by hand. However, this difficulty can be adverted with the inclusion of computational intelligence as a design companion. In the behaviour based controller, only generic representations were specified for each behaviour component. Each component is subsequently trained using genetic algorithm. For example, a potential field representation was used for the steering control but it was not specified beforehand whether the interactions were attractive or repulsive in nature. The evolved controller exhibited attractive forces which was necessary for good performance in the game.

The behaviour based controller consists of five main components, illustrated in Fig. 2. Components that form the basic driving behaviour are organized at the lower base level while tactical behaviours are organized at the higher first level. The first level can be said to subsume the base driving level. The individual component will now be discussed in further detail.

3.1. Force field trajectory

The first is a trajectory planning mechanism inspired by the interaction between charged particles in space. Potential field

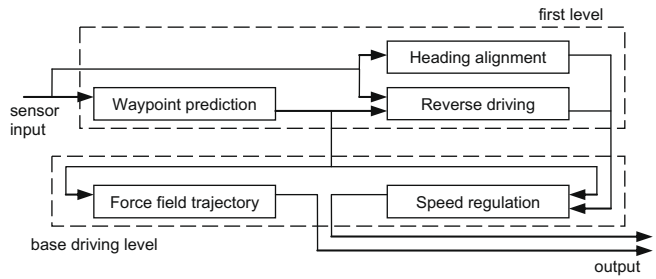

Fig. 2. Overview of behaviour based controller.

methods are widely used in the field of robotics due to its simplicity (Ren, McIsaac, Patel, & Peters, 2007). Every foreign particle in the playing field, namely the car belonging to the opponent, the current waypoint and the next waypoint, induces either an attractive or repulsive field on the game field. At any point on the game arena, the controller tries to align its car in the direction of the local induced vector field. As such, the controller car will move along the resultant field lines induced by the interaction of these charged particles. However, these field lines only indicate the steering path and not the driving speed. The field equation for the particles in the game arena is defined in (1)

$$\vec{E}_i = q_i r^{p_i} \hat{r}, \quad i = \{o, wp1, wp2\} \tag{1}$$

where o is the opponent vehicle, $wp1$ is the current waypoint, $wp2$ is the next waypoint, E_i^V is the field vector induced by the point particle i, q_i is the charge of particle i, r is the distance from the particle with charge q_i and the evaluation point, p_i is the power factor of the distance r and $\hat{r}$ is the unit vector pointing from the particle with charge q_i to the evaluation point. The variables q_i and p_i for the opponent car, the current waypoint and the next waypoint were optimized using genetic algorithm. The controller's car was considered a positive point charge in calculations in order to evaluate the resultant force exerted on the controller car. There is no constraint on the evolved variable so it is entirely possible that the results may turn out to be other than expected.

3.2. Speed regulation

The force field trajectory component determines only the driving path of the car and not the driving speed. Hence, a speed regulating function, which constitutes the second base component of this controller, is introduced to specify the driving speeds along the steering trajectory. An important driving feature which is crucial for the performance of the controller is the ability to stop at a specific position in the playing field. Although it may seem counterintuitive to stop in a racing game, this action becomes necessary when we consider going for the next waypoint instead of the current waypoint. Suppose the opponent is going to reach the current waypoint first, it makes sense to head towards the next waypoint directly. But in the situation that the controller car arrived at the next waypoint before the opponent can reach the current waypoint, the controller will then need to stop the car at the next waypoint and wait until it becomes activated. The equation for the speed regulating function is defined in (2)

$$\text{Speed} = a \times \tanh(b \times r + c) + d \tag{2}$$

where r is the distance to the destination and a, b, c and d are parameters characterizing the speed regulation function. The four parameters were optimized using genetic algorithm. The hyperbolic tangent function was chosen because of its general shape. The tapering of its outputs at high values of r is analogous to the notion that the car should cruise at a constant speed at far distances from its destination (i.e. the cruising speed should not increase indefinitely with distance). Additionally, the steep gradient around the origin is analogous to deceleration when it is near the destination. The values a, b, c and d serves to shape the hyperbolic tangent function to one most desirable for this car racing simulation. There are no constraints that the function need to pass through the origin or that it should be positive or negative.

3.3. Reverse driving

A desirable driving feature for this type of point to point race is the ability to drive in reverse. A human player who just started playing the game would very soon realize that if a waypoint is situated at close proximity but directly behind the car, it is faster to simply reverse the car towards it. This type of behaviour was not present in the HeuristicSensibleController (HSC). As a result, the HSC was often seen, much to the frustration of the human observer, to take a non-optimal U-turn to approach a waypoint behind it. Moreover, in the process of performing the U-turn, the controller often underestimated the turning radius and became trapped in an endless orbit about the waypoint. To rectify such behaviours, a reverse driving threshold variable is introduced to the behaviour based controller. The angle towards the destination was included to determine whether to drive forwards or reverse in a given situation. If the angle is within a given threshold, the speed function will be negated and the controller will reverse the car towards the destination instead. The threshold parameters were evolved using genetic algorithm.

3.4. Waypoint prediction

The fourth component is a predictive module that chooses which waypoint to compete for. By observing the state of the game area, the controller predicts which car will reach the current waypoint first. In the event that the opponent is predicted to be faster to the current waypoint, the controller should then direct the car towards the next waypoint instead and vise versa. This predictive module sits on top of the base driving layer and is capable of augmenting the inputs to the base driving layer. The pseudo-code for the waypoint prediction component is as follows.

$$\begin{aligned}
&\textit{if} \quad \left(\frac{distance(c, wp1)}{distance(o, wp1)} < 1\right) \\
&\qquad \textit{return } c; \\
&\textit{elseif} \\
&\qquad \left(\frac{distance(c, wp1)}{\text{speed}(c, wp1)} \times \frac{\text{speed}(o, wp1)}{distance(o, wp1)} < 1\right) \\
&\qquad \textit{return } c; \\
&\textit{else} \\
&\qquad \textit{return } o; \\
&\textit{end}
\end{aligned}$$

where distance (i,j) refers to the Euclidean distance measured between point i and point j, speed(i,j) refers to the magnitude of the vector component of the speed of vehicle i along the direction from point i towards point j, c is the car controlled by the controller calling this function, o is the opponent vehicle and $wp1$ is the current waypoint.

During each time step, the waypoint prediction system determines which vehicle will reach the current waypoint first. If the opponent vehicle will reach first, then the controller will direct both the force field trajectory and the speed regulator towards the next waypoint instead. The waypoint prediction system is

designed using simple domain knowledge and reasoning. First, the component speed of each vehicle in the direction of the current waypoint is calculated using vector scalar product. Next, the pseudo-code is used to determine which vehicle will reach the current waypoint earlier. In essence, the controller will drive the car towards the current waypoint if it was nearer to it than the opponent vehicle is. Even if it is further away compared to the opponent, it will still drive towards the current waypoint if it takes a shorter time to reach there based on the instantaneous component speed of each vehicle calculated in the previous step.

3.5. Heading alignment

When the controller is driving the car towards the current waypoint, it may be advantageous to align its heading with that of the next waypoint just as it passes through the current waypoint. This will allow for a smoother driving line from the current waypoint to the next and increase the likelihood of reaching the next waypoint before its opponent. Currently, this behaviour is only implemented for the forward driving direction. The pseudo-code for the heading alignment behaviour is as follows:

$$\begin{array}{l} \textit{if} \quad \left(\begin{array}{l} (distance(c, wp1) < k_1) AND \\ (-k_2 < angle(c, wp2) < k_2) AND \\ (speed(c) > k_3) \end{array} \right) \\ \qquad return\ true; \\ else \\ \qquad return\ false; \\ end \end{array}$$

where distance (i,j) refers to the Euclidean distance measured between point i and point j, $angle(i,j)$ refers to the angle in radians of point j from point i, speed(i) refers to the current instantaneous speed of object i, c is the car controlled by the controller calling this function, $wp1$ is the current waypoint and $wp2$ is the next waypoint. Three variables k_1, k_2 and k_3 defines the thresholds of the activation of this function and were evolved using genetic algorithm. If the function returns true, the car will be steered to face the next waypoint instead of the current waypoint. Otherwise, the car will continue on its current path.

4. Experiments

A genetic algorithm (GA) of population size of 30, running for 100 generations was used as a training method for the behaviour based controller. Each individual is evaluated against the HeuristicSensibleController (HSC) for five rounds of competition, followed by five rounds of competitive coevolution against a random elite individual from the population. The fitness function was defined as the number of waypoints the individual collected averaged over the 10 rounds of game play. Each game is played for 1000 time steps. Elitism was implemented by retaining the best four individuals of each generation. The same four elite individuals also participate as coevolution opponents during the fitness evaluation of other individuals in the population. Each chromosome for the behaviour based controller was encoded with a total of 14 real valued variables, six from force field component, four from speed regulation component, one variable which encode the threshold for reversing driving, and three variables for the heading alignment component. All variables were initialized by a random Gaussian distribution with a mean of 0 and a variance of 1. The crossover and mutation operate will be discussed in further details next.

4.1. Crossover operator

Each chromosome was encoded using real values so the crossover operator must be designed to work with real numbers. The pseudo-code for the crossover operator is as follows:

$$\begin{array}{l} for\ each\ pair\ of\ genes \\ \quad if(random[0,1] < crossover\ rate) \\ \qquad weight = random[0,1]; \\ \qquad offspring = weight \times parent1 + (1 - weight) \times parent2; \\ \quad end \\ end \end{array}$$

The variable weight gives a random emphasis on the gene from one of the parent. For example, if the weight was 0.5, the result would be the average value of the genes from both parents. However, if the weight was 0.8, then the offspring would inherit 80% of the gene from parent 1 and the remaining 20% of the gene from parent 2.

The effect of the crossover rate was investigated by varying its value from 0.0 to 1.0 in steps of 0.2 while the value of the mutation rate was arbitrarily set to 0.2. The results of varying the crossover rate are plotted in Fig. 3. The inclusion of the crossover operator generally produced better results compared to when the crossover rate was set to 0. In all cases, the results converge after about 30 generations.

4.2. Mutation operator

A Gaussian perturbation with a mean value of 0 and variance of 1 was used as the mutation operator. For each gene, the Gaussian perturbation was applied with a probability given by the mutation rate.

The effect of the mutation rate was investigated by varying its value from 0.0 to 1.0 in steps of 0.2 while the value of the crossover rate was arbitrarily set to 0.8. The results of varying the mutation rate are plotted in Fig. 4. In particular, the case of mutation rate = 0.0 converged to a local minimum. The value of mutation rate = 0.2 was observed to be most optimal amongst the different choices of mutation rate. The rate of convergence was also around 30 generations.

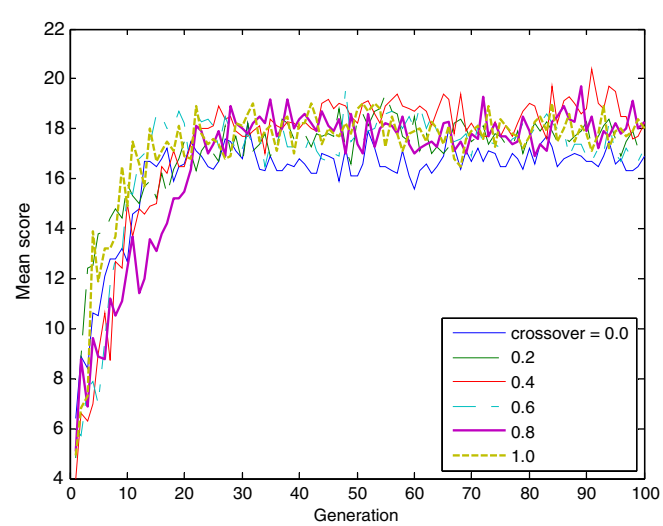

Fig. 3. Effects of varying crossover rate; mutation rate fixed at 0.2.

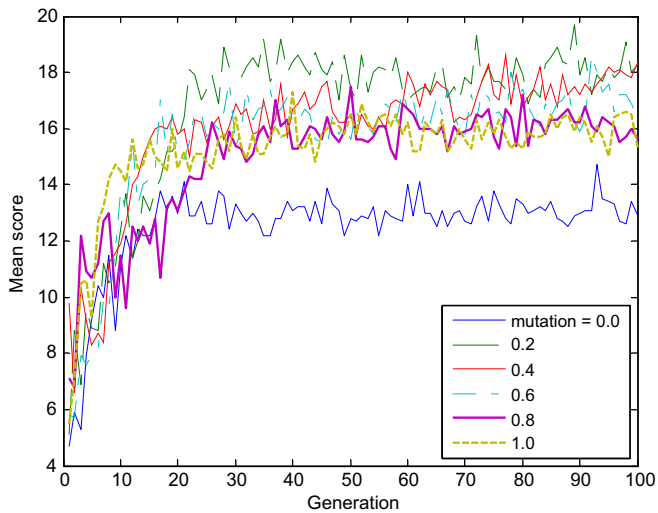

Fig. 4. Effects of varying mutation rate; crossover rate fixed at 0.8.

4.3. Simulation results

Based on the prior investigation, a crossover rate of 0.8 and a mutation rate of 0.2 were chosen to evolve the behaviour based controller. The best individual from the last generation was examined to investigate the characteristics of the evolved behaviour based controller. The five components are described as follows. The force field trajectory component provides some guidelines for the controller to plans its path from the current waypoint to the next. The speed regulating component defines the acceleration, deceleration and stopping behaviour. The reverse threshold decides when is the best time to reverse towards a target. The waypoint prediction directs the car to the next waypoint if the current waypoint cannot be reached before the opponent. Finally, the heading alignment decides when it is best to turn towards the next waypoint. The results and performance of the behaviour based controller will be discussed from two perspectives in this section. Firstly, the white box nature of the behaviour based controller allowed for the analysis of the evolved values. These evolved parameters will be examined to gain a better appreciation of the behaviour based controller. Secondly, the functionality of the individual components will be examined for their impact on the overall performance of the controller.

4.4. Evolved parameters analysis

The evolved values for the force field trajectory component are presented in Table 1 and the field strength is plotted against distance r in Fig. 5a. All forces acting on the car are attractive in nature since all the controlled car is assumed to be a positive point charge and the evolved q_i values turned out to be negative. The field strength of the current waypoint is at least 10 times larger than that of the opponent car and the next waypoint within the range of the game area. This implies the controller car is strongly attracted to the current waypoint while the effects from the opponent car and the next current are minimal. Therefore, the controller will direct the car towards the current waypoint regardless of its distance. This result is similar to our intuition that is to steer in the direction of the destination. Additionally, this way of steering is applicable both when driving forward and in reverse. The value of q_{other} was initially expected to be repulsive in nature as it seemed sensible to avoid collisions with the opponent, but this controller evolved a new strategy that is to intentionally collide with the opponent when sufficiently near. This is because the reverse driving component allows the controller to recover quickly after a collision by simply driving in the direction facing the current waypoint. This becomes an advantage if the opponent only drives in one direction like the HeuristicSensibleController. In general, the output trajectory is an approximate straight line towards the current waypoint with minor disturbances from the other vehicle and next waypoint.

Table 1
Evolved force field trajectory parameters of best individual.

i	*other*	*wp*1	*wp*2
q_i	−0.02803	−0.896679	−0.063289
p_i	−0.10153	−0.08817	0.377045

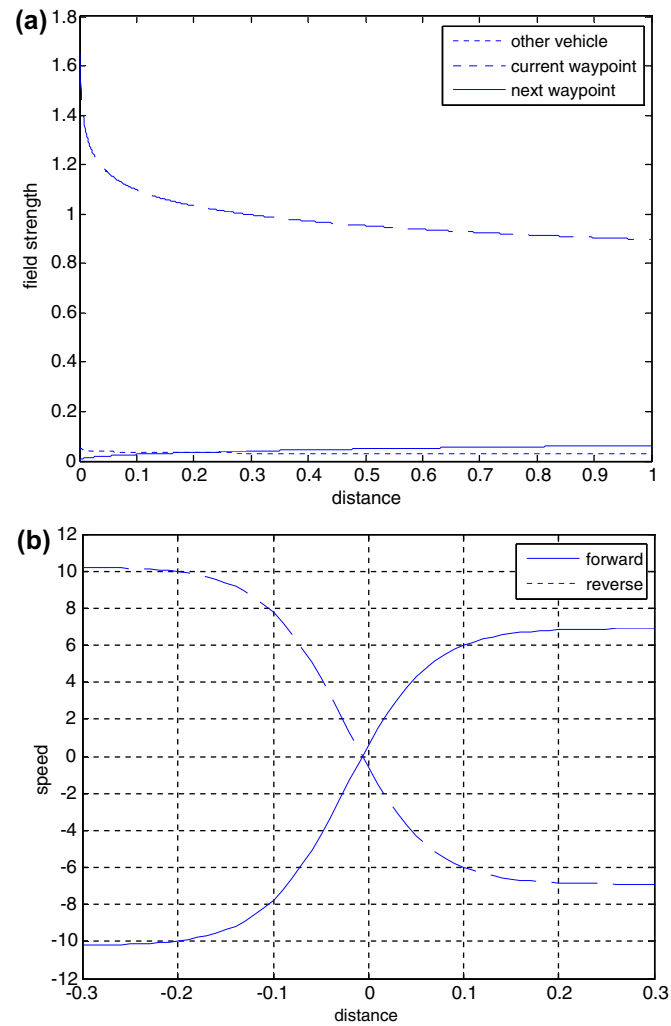

Fig. 5. Graph of evolved parameters for behaviour based controller for (a) field strength against distance from particle and (b) desired driving speed against distance from destination.

From Fig. 5b, it is observed that a driving speed limit of 7 units per time step is imposed by the speed regulating function on the controller vehicle. For distances less than 0.2 to the destination, the car switches to rapid deceleration before coming to a halt at the destination point. The negative values of distance r were not used in the actual game as distances were strictly positive. Traveling within these speed limits, the car does not skid and hence does not exhibit any advanced driving techniques that require skidding.

Table 2
Evolved heading alignment parameters of best individual.

Parameter	Value
k_1	0.0551
k_2	2.5513
k_3	4.3076

An additional parameter was included to determine whether to drive forwards or in reverse in a given situation. If the angle to the destination is within the threshold stated by the parameter, the speed regulating function will be negated and the controller will reverse the car towards the destination instead. The final evolved value of this parameter is 1.897 radians. This implied that if the waypoint is located within a span of 142.5° centered directly behind the car, the controller will drive in reverse towards the destination. Additionally, the speed plots of both forward and reverse in Fig. 5b passes very close to the origin. In the actual game, this was sufficient to stop the vehicle exactly at its desired destination. This was observed in two player competition where the behaviour based controller often stops the car at the next waypoint while waiting for the current waypoint to be passed.

The evolved parameters for the heading alignment component are summarized in Table 2. At a distance of 0.0551, the behaviour based controller is likely to be in a state of deceleration according to Fig. 5b. At this distance, if the next waypoint is within −2.5513 to 2.5513 radians or within −146.2° to 146.2° centered in front of the car and the speed of the car is more than 4.3076 units per time step, the heading alignment component will steer the car to face the next waypoint while continuing its approach toward the current waypoint. The speed threshold ensured that the car does not miss its current waypoint while trying to steer towards the next. This enabled the behaviour based controller to put itself in a better position to accelerate towards the next waypoint once it passes the current one.

4.5. Behaviour components analysis

Five behavioural components were implemented in the behaviour based controller. Each of these components can be optionally activated or deactivated giving a total of 32 combinations. In order to appreciate the impact of each component on the overall performance of the behaviour based controller, all combinations of the controller is benchmarked against the case of solo run, competition against the HeuristicSensibleController (HSC) and against the HeuristicCombinedController (HCC) in Table 3. The combination of components activated is abbreviated under the column behaviour set in the format $X_1X_2X_3X_4X_5$ where X_1 represents waypoint prediction, X_2 represents force field trajectory, X_3 represents speed regulation, X_4 represents reversing and X_5 represents heading alignment. In the deactivated state, the force field trajectory is replaced with intuitive steering that steers in the direction of the destination and the speed regulation threshold is set to seven independent of distance. As a reference, the results for HSC running the same benchmark are also listed on row 0.

It was observed that component X_1 (waypoint prediction) had insignificant impact during the solo run. This was to be expected as there was no opponent in the solo case and performance was entirely dependent on driving skills. In two player situations, waypoint prediction generally improves the controller performance as evident when comparing pair wise between rows 1 and 16 with their counterpart from rows 17 to 32. This implied that waypoint prediction mainly contributed to improvements in competitive games.

Although pair wise comparisons for X_2 (force field trajectory) from Table 3 did not give a clear indication of its advantage, its value could be better appreciated visually. In general, the driving line traced by the force field trajectory was smoother than that of intuitive steering, resulting in a more realistic driving style rather than a mechanic one that constantly jerks left and right in order to keep on a straight path.

Table 3
Comparative studies of behaviour set.

Row	Skill	Solo	vs HSC	vs HCC
0	HSC	13.730	10.528	10.726
1	00000	11.424	11.674	9.678
2	00001	10.802	11.668	9.660
3	00010	14.350	12.880	12.654
4	00011	14.016	12.766	12.580
5	00100	16.422	11.894	14.700
6	00101	17.178	12.236	15.256
7	00110	20.092	13.154	16.746
8	00111	20.974	13.272	16.858
9	01000	10.988	11.572	9.292
10	01001	11.308	11.684	9.774
11	01010	14.354	13.168	12.154
12	01011	13.854	13.044	12.324
13	01100	16.262	11.810	15.176
14	01101	17.532	11.922	15.204
15	01110	20.168	13.478	16.920
16	01111	21.006	13.388	16.938
17	10000	11.396	14.728	10.524
18	10001	10.926	14.828	10.574
19	10010	14.288	16.144	13.692
20	10011	13.904	16.216	13.442
21	10100	16.598	15.966	18.292
22	10101	17.472	16.184	18.970
23	10110	20.084	17.252	19.740
24	10111	20.776	17.646	20.116
25	11000	11.142	14.600	10.282
26	11001	11.248	14.672	10.658
27	11010	14.640	15.942	13.302
28	11011	13.646	16.116	13.374
29	11100	16.596	16.006	18.384
30	11101	17.182	16.046	18.316
31	11110	20.224	17.148	19.742
32	11111	21.074	17.758	20.026

By comparing pair wise of rows 1 and 5 and other corresponding pairs that compare X_3 (speed regulator), it was observed that the speed regulator improves performance both in the solo run and against HCC. This was mainly due the speed regulator slowing down the car near its destination and hence greatly reduced the occurrence of orbiting and this translates into higher scored points for the controller. The difference was even greater when the speed regulator worked in combination with the waypoint predictor as it enabled the controller to stop the car at the next waypoint when waiting for its opponent to pass the current waypoint.

In the comparison for component X_4 (reversing) the general trend observed was that activating the reverse driving feature improves performance in all three cases of solo run and competitive games (i.e. rows 1 and 3). The difference was more significant when it was used in combination with the speed regulator (i.e. rows 1 and 7). This was because the speed regulator slows down the car at its destination, making the changing of direction smoother and less time consuming. In visually observed games, the reversing behaviour was also seen as the main contributor to collision recovery as the controller was able to drive in whichever direction that was facing its destination.

The independent effects of component X_5, heading alignment, can be observed by comparing the column solo on rows 5 and 6 and rows 7 and 8. The heading alignment mainly improves the performance of solo games with used in conjunction with the speed regulation component. This is likely due to the fact that all parameters were evolved simultaneously. As a result, the genetic algorithm successfully exploited this collaboration between the two

components. The heading alignment behaviour also improved results in two player games but the improvement was of a smaller margin.

The analysis of the individual components of the behaviour based controller has also provided some insights on how the controller can be improved in the future. These suggestions will be discussed here for possible implementation in the future. An inefficiency of the speed regulator is that it treats the current waypoint and next waypoint indifferently, which turns out to be suboptimal. Although the controller needs to stop the car at the next waypoint, the same cannot be said about the current waypoint. In a race to the current waypoint, there is no need to slow to a complete halt at the waypoint. Instead, crossing the current waypoint with moderate and controllable speed can be considered a better choice. Hence, separate speed regulation models for the current and the next waypoint could improve the performance of the controller. To further complicate matters, the forward acceleration is twice that of the reverse acceleration. This meant the speed regulation can be further broken down into forward and reverse components rather than simply negating one component to get the other. While these incremental improvements are beneficial, including these features would require detailed analysis of the game dynamics. Another method of implementation may be to divide the speed regulation component into four separate functions and employ GA to optimize the function parameters without the need to analyze the game dynamics.

5. Results and discussion

The comparative studies so far were conducted under a controlled environment where simple heuristic controllers were used. To further substantiate its performance, the behaviour based controller was tested against the top five entries of the Simulated Car Racing Competition held during the 2007 IEEE Congress of Evolutionary Computation (CEC) to test its generalization performance against previously unseen opponents. In the competition, each entry was ranked using a competition benchmark known as CompetitionScore. However, the controller with the highest benchmark score at this point is not necessarily the winner of the competition. The winner of the competition is the winner of a final round robin tournament. The tests in this section will be conducted in a similar manner. The top five controllers and the behaviour based controller will run the benchmark CompetitionScore and their scores and the time taken for simulation will be recorded. Thereafter, all six controllers will take part in a round robin tournament and the scores of each pairings will be recorded.

5.1. CompetitionScore benchmark

The CompetitionScore metric is the benchmark metric used to rank the submitted controllers before the final tournament. This metric is defined as the mean fitness of 500 trials in each of these three scenarios: solo trial, versus HeuristicSensibleController (HSC) and versus HeuristicCombinedController (HCC).

Table 4
Comparative results of CompetitionScore of behaviour based controller against top five controllers.

Controller	CompetitionScore	Simulation time in seconds
Controller A	20.539 (0.0416)	8450.1 (308.27)
Controller B	16.551 (0.0509)	2683.5 (217.95)
Controller C	19.176 (0.0388)	20.3 (2.31)
Controller D	18.933 (0.0662)	473.8 (73.86)
Controller E	18.797 (0.0757)	66.6 (2.675)
Behaviour based	**19.558 (0.0536)**	**17.5 (2.42)**

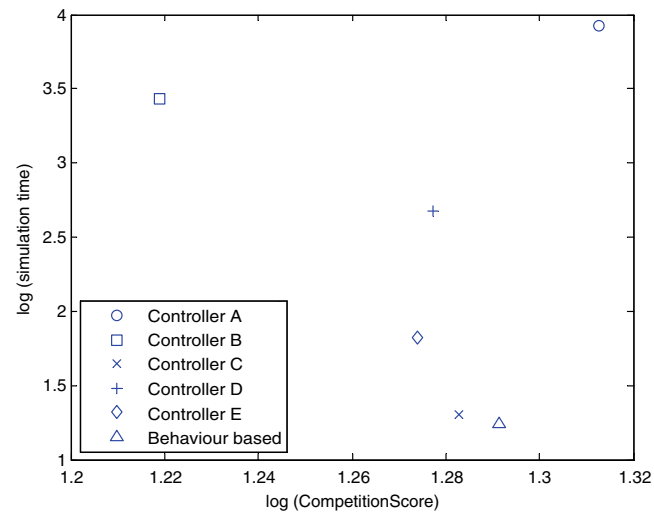

Fig. 6. Pareto plot pf $\log_{10}$ (simulation time) vs $\log_{10}$ (CompetitionScore). Low simulation time and high CompetitionScore preferred.

In order to achieve a high CompetitionScore, a controller needs to perform well on its own (i.e. solo run), as well as against a weak (i.e. HSC) and an intermediate (i.e. HCC) controller.

The mean scores and times of the CompetitionScore benchmark, and their standard deviations in parentheses, are presented in Table 4. Controllers A, B, C, D and E are the controllers ranked 1, 2, 3, 4 and 5 respectively on the competition website (Togelius & Lucas, 2007). Since there are two metrics of comparison, the results are plotted in the form of a Pareto dominance diagram in Fig. 6. for better visualization. The axes are shown in logarithmic form due to the presence of very large and very small differences in simulation times. The behaviour based controller is highlighted in bold. The performance of the behaviour based controller will be discussed in this section in terms of its score as well as its computation efficiency. The results were averaged over 10 runs.

The behaviour based controller scored 19.558 in the benchmark and was ranked second amongst the six controllers. In terms of computation efficiency, the behaviour based controller was the most efficient controller, completing the benchmark in 17.5 s. Comparatively, the top scoring controller (Controller 1) took an average of 8450.1 s to complete the benchmark, or (8450.1/17.5≈) 482 times slower than the behaviour based controller. In real time games, game AI is usually allocated a very small CPU cycle budget (a large portion goes to rendering graphics), making computational intensive algorithms less attractive and hence the need for an efficient game controller. This makes the behaviour based controller a more suitable candidate for implementation in real time games.

In order to appreciate how the controllers performed in terms of both performance metrics simultaneously, the Pareto ranking of the controllers are considered in Fig. 6. In a two dimensional Pareto diagram, each axes represents a performance metric. In our situation, the two performance metrics are, simulation time on the vertical axes, and CompetitionScore on the horizontal axes. A low simulation time and a high CompetitionScore are desired. A controller is said to be dominated if there is another controller that outperforms it in both the performance metrics. Conversely, a controller is said to be non-dominated if there are no other controllers that outperforms it in both performance metrics. A controller is then given a Pareto rank defined by Eq. (3)

$$rank(i) = 1 + n_i \tag{3}$$

Table 5
Pareto ranks of top five controllers and behaviour based controller.

Controller	Pareto rank
Controller A	1
Controller B	5
Controller C	2
Controller D	3
Controller E	3
Behaviour based	**1**

where n_i is the number of controllers dominating the individual controller i. For example, Controller E is dominated by Controller C and the behaviour based controller, hence $n_E = 2$ and rank(E) = 3. The minimum Pareto rank is 1. The Pareto ranks of the controllers are summarized in Table 5.

It was observed from Table 5 that, according to Pareto optimality, the behaviour based controller and controller A obtained the highest Pareto rank of 1 amongst the six controllers. This meant that neither of the two controllers was completely dominant over the other controller. Controller A had obtained a higher CompetitionScore compared to the behaviour based controller while the latter obtained a lower simulation time. In order to gain better insights to the differences between the two controllers, another performance indicator was required.

5.2. Round robin tournament

In CompetitionScore, all the controllers were tested on their own, and against two benchmark controllers. These benchmark controllers could be used during training to obtain a high CompetitionScore value. This benchmark does not test their generalization performance against unseen opponents. As such, generalization performance could be used to further distinguish the controllers. To do this, the six controllers were tested against one another in a round robin tournament. The scores from the round robin tournament are recorded in Table 6 and the t-values are also listed below each pair of scores. Each pair of controllers played 500 games against each other. The results are summarized in Table 7, sorted first by the number of wins, then by number of draws, then by number of losses, and finally by the total points scored. The behaviour based controller is highlighted in bold in both tables.

From Table 6, it was observed that the behaviour based controller obtained a higher mean score (significant at 0.05 level) than its opponent against all five other controllers. Controller A, which had a Pareto ranking of 1, lost against the behaviour based controller and drew (difference in score not significant at 0.05 level) its game against Controller C. It was also noted that Controller B (Pareto ranking 5) drew its game against Controller E (Pareto ranking 3). From Table 7, the behaviour based controller was the best performing controller with five wins and it also scored the highest total number of points in the tournament. This implied that the behaviour based controller exhibit the best generalization performance amongst the 6 controllers being tested. This result also distinguished the behaviour based controller, from the similarly Pareto ranked 1 Controller A, as the better performing controller. Generalization performance is important in the context of games because of its wide array of varied customer base. If the game AI does not perform reliably well against players with different playing styles, its perceived quality will be degraded.

Table 7
Consolidated results for tournament of top five controllers.

Controller	Win/draw/loss	Total points scored	Total points against
Behaviour based	**5/0/0**	**96.436**	**84.934**
C	3/1/1	95.104	85.898
A	3/1/1	92.456	81.082
D	2/0/3	89.668	83.368
E	0/1/4	69.372	85.514
B	0/1/4	62.168	84.408

The behaviour based controller was able to demonstrate its generalization performance and computation efficiency through this experiment. The behaviour base controller was Pareto non-dominated in terms of CompetitionScore and simulation time, and also top ranked in the round robin competition amongst the six controllers. For future work, the extension of the behaviour based controller to incorporate learning behaviours will be considered. The current behaviour based controller is a static controller which does not learn as it plays the game. As such, good human players will be able to learn its driving patterns and develop counter strategies that can reliably win against it. In fact, this author has a strategy that can win against the behaviour based controller in direct competition. In order to consistently offer a challenging game playing experience and hence upgrade the entertainment value of the game for the human player, an adaptive controller that is capable of in-game learning is desirable, but this must also work within the computational efficiency requirement of real time games.

Table 6
Results for direct competition between top five controllers and behaviour based controller.

Controller	A	B	C	D	E	**Behaviour based**
A	–	17.256 (2.928)–11.404 (2.760)	19.188 (2.424)–19.128 (2.110)	18.234 (2.715)–17.344 (2.258)	18.890 (2.640)–13.736 (2.455)	**18.888 (2.417)–19.470 (2.060)**
t-Value	–	32.52	0.42	5.64	31.97	**−4.10**
B	–	–	12.912 (2.257)–18.510 (2.102)	13.198 (2.369)–18.142 (2.202)	11.960 (2.360)–11.796 (2.435)	**12.694 (2.300)–18.704 (2.145)**
t-Value	–	–	−40.59	−34.18	1.08	**−42.73**
C	–	–	–	19.374 (1.923)–18.462 (2.346)	18.522 (2.059)–15.502 (2.281)	**19.570 (2.137)–19.834 (1.965)**
t-Value	–	–	–	6.72	21.98	**−2.03**
D	–	–	–	–	17.284 (2.342)–12.992 (2.799)	**18.436 (2.469)–19.570 (2.230)**
t-Value	–	–	–	–	26.30	**−7.62**
E	–	–	–	–	–	**15.346 (2.321)–18.858 (2.204)**
t-Value	–	–	–	–	–	**−24.54**
Behaviour based	–	–	–	–	–	–
***t*-Value**	–	–	–	–	–	–

6. Conclusion

A real time car racing controllers based on the behaviour based design methodology was proposed in this paper. The disadvantage of developing a behaviour based controller is its requirement for hand designed components. This paper ultilized genetic algorithm to complement the design of the individual behavioural components. Five behaviour components were evolved using genetic algorithms. In the analysis of the evolved behaviours, it was observed that the genetic algorithm successfully exploited some collaboration between the different behaviour components which may have gone unnoticed if it was designed by hand. The best evolved controller was benchmarked against the top five controllers from the IEEE CEC 2007 Simulated Car Racing competition to test its generalization performance against unseen opponents. The controllers were evaluated based on their scores using the ComputationScore benchmark, the simulation time taken, and their generalization performance in a round robin tournament against one another. The behaviour based controller scored the second highest in CompetitionScore but was about 482 times faster than the top scoring controller. In the round robin tournament, the behaviour based controller was able to demonstrate its better generalization capability and outperformed all the other five controllers. Its better computation efficiency and generalization performance makes the behaviour based controller a more suitable candidate for implementation in real time games. A limitation of the current controller design is its static nature. Therefore, a possible area for extension is developing an adaptive controller that have the potential to learn the driving patterns of its static opponents and develop counter strategies against them, hence improving its performance and robustness.

Acknowledgement

This research was supported by Academic Research Fund (AcRF) under Project No.: R-263-000-480-112.

References

Braathen, S., & Sendstad, O. J. (2004). A hybrid fuzzy logic/constraint satisfaction problem approach to automatic decision making in simulated game models. *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics, 34*(4), 1786–1797.

Brooks, R. A. (1986). A robust layered control system for a mobile robot. *IEEE Journal of Robotics and Automation, 2*(1), 14–23.

Bryant, B. D., & Miikkulainen, R. (2003). Neuroevolution for adaptive teams. In *Proceedings of IEEE congress on evolutionary computation* (Vol. 3, pp. 2194–2201).

Bryson, J. J. (2002). The behavior-oriented design of modular agent intelligence. *Agent Technologies, Infrastructures, Tools, and Applications for E-Services*, 61–76.

Buro, M., & Furtak, T. (2003). RTS games as test-bed for real-time AI research. In *Proceedings of the seventh joint conference on information science* (pp. 481–484).

Chaperot, B., & Fyfe, C. (2006). Advanced artificial intelligence techniques applied to a motocross game. *Computing and Information Systems, 10*(2), 27–31.

Chellapilla, K., & Fogel, D. B. (2001). Evolving an expert checkers playing program without using human expertise. *IEEE Transaction of Evolutionary Computation, 4*, 422–428.

DeSouza, G. N., & Kak, A. C. (2004). A subsumptive, hierarchical, and distributed vision-based architecture for smart robotics. *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics, 34*(5), 1988–2002.

Fernlund, H. K. G., Gonzalez, A. J., Georgiopoulos, M., & DeMara, R. F. (2006). Learning tactical human behavior through observation of human performance. *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics, 36*(1), 128–140.

Forbus, K., & Laird, J. (2002). AI and the entertainment industry. *IEEE Intelligent Systems, 17*(4), 15–16.

Ghoneim, A., Abbass, H., & Barlow, M. (2008). Characterizing game dynamics in two-player strategy games using network motifs. *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics, 38*(3), 682–690.

Lucas, S. M., & Kendall, G. (2006). Evolutionary computation and games. *IEEE Computational Intelligence Magazine*, 10–18.

Miikkulainen, R., Bryant, B. D., Cornelius, R., Karpov, I. V., Yong, K. O., & Stanley, C. H. (2006). Computational intelligence in games. Computational intelligence: Principles and practice. *IEEE Computational Intelligence Society*, 155–191.

Nakashima, T., Udo, M., & Ishibuchi, H. (2004). A fuzzy reinforcement learning for a ball interception problem. Lecture notes in computer science, RoboCup 2003: Robot Soccer World Cup VII (Vol. 3020, pp. 559–567).

Nakashima, T., Takatani, M., Udo, M., & Ishibuchi, H. (2004). An evolutionary approach for strategy learning in robocup soccer. *IEEE International Conference on Systems, Man and Cybernetics, 2*, 2023–2028.

Quek H. Y., & Goh, C. K. (2007). Adaptation of iterated prisoners dilemma strategies by evolution and learning. In *Proceedings of 2007 IEEE symposium on computational intelligence and games* (pp. 40–47).

Ren, J., McIsaac, K. A., Patel, R. V., & Peters, T. M. (2007). A potential field model using generalized sigmoid functions. *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics, 37*(2), 477–484.

IBM Research, Deep Blue. <http://www.research.ibm.com/deepblue/> Retrieved 15.05.07.

Schaeffer, J. (1997). *One jump ahead: Challenging human supremacy in checkers*. Springer-Verlag.

Schaeffer, J. (2001). A gamut of games. *Artificial Intelligence Magazine, 22*(3), 29–46.

Sengers, P. (1998). Do the thing right: An architecture for action expression. In *Proceedings of the second international conference on autonomous agents* (pp. 24–31).

Spronck, P. (2005). Adaptive game AI. PhD thesis, Maastricht University Press.

Spronck, P., Ponsen, M., Sprinkhuizen-Kuyper, I., & Postma, E. (2006). Adaptive game AI with dynamic scripting. *Machine Learning, 63*(3), 217–248.

Stanley, K. O., Bryant, B. D., Karpov I., & Miikkulainen, R. (2006). Real-time evolution of neural networks in the NERO video game. In *Proceedings of the twenty-first national conference in artificial intelligence* (pp. 1671–1674).

Stone, P., Sutton, R. S., & Kuhlmann, G. (2005). Reinforcement learning for RoboCup-soccer keepaway. *Adaptive Behaviour, 13*(3), 165–188.

Togelius, J., & Lucas, S. M. (2005). Evolving controllers for simulated car racing. In *Proceedings of IEEE congress on evolutionary computation* (Vol. 2, pp. 1906–1913).

Togelius, J., & Lucas, S. M. (2006). Evolving robust and specialized car racing skills. In *Proceedings of the IEEE congress on evolutionary computation* (pp. 1187–1194).

Togelius, J., & Lucas, S. M. (2006b). Arms races and car races. *Lecture Notes in Computer Science, Parallel Problem Solving from Nature, 4193*, 613–622.

Togelius, J., & Lucas, S. M. (2007). IEEE CEC 2007 Car Racing Competition. <http://www.julian.togelius.com/cec2007competition/> Retrieved 18.11.07.

Tozour, P. (2002). *The evolution of game AI*". *AI game programming wisdom*. Charles River Media, Inc.. pp. 3–15.

Yannakakis, G. N., & Hallam, J. (2004). Evolving opponents for interesting interactive computer games. In *Proceedings of 8th international conference on the simulation of adaptive behavior* (pp. 499–508).

Yannakakis, G. N., & Hallam, J. (2005). A generic approach for generating interesting interactive pac-man opponents. In *Proceedings of 2005 IEEE symposium on computational intelligence and games* (pp. 94–101).